GUÍA DEL ALUMNO

Ver lo que pasa. Verificar lo que escribes.

La Tienda de cursos, revisada: **Vue DevTools, vue-tsc, ESLint, Prettier y asistentes de IA en el editor.**

Herramientas de desarrollo
Vue 3.5 · TypeScript · ESLint 10

Oscar Alvarado
Frontend Tech Lead

Universidad San Martín de Porres
Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

La misma Tienda, vista desde adentro

Seguimos con **la Tienda de cursos** de la sesión 3. Hoy casi no escribimos código nuevo: aprendemos a **mirarla por dentro**, a **comprobar que está bien** y a **decidir cuándo dejar que una IA escriba**. Antes de cada tema está la **teoría**: qué es la herramienta, para qué sirve y un diagrama de cómo funciona, el mismo de las slides. Después, los pasos para probarla sobre la Tienda, diciendo **dónde** va cada cosa.

1
Ver
Vue DevTools · depurar estado y reactividad

2
Verificar
vue-tsc · ESLint (flat config) · Prettier

3
Generar
asistentes de IA · contexto + contrato · revisión

La clase en tres momentos

```
La misma Tienda, tres momentos

1 · VER        Vue DevTools
               componentes · estado · eventos

2 · VERIFICAR  vue-tsc · ESLint · Prettier
               npm run verificar

3 · GENERAR    un asistente de IA
               FiltroCategoria.vue
```

Cada herramienta responde una pregunta distinta: las **DevTools**, *¿qué estado tiene mi app ahora?*; **vue-tsc**, *¿los tipos cuadran?*; **ESLint**, *¿hay errores o malas prácticas?*; **Prettier**, *¿se ve como el resto?*; la **IA**, *¿puedo escribirlo más rápido?* (y siempre se revisa).

Lo que necesitas instalado

- **Node.js 22.18 o superior** (comprueba con `node -v`).
- **VS Code** con la extensión **Vue (Official)**. Opcional: las extensiones **Vue.js devtools** (navegador) y **Prettier** (VS Code).
- Un **asistente de IA** en el editor, el que prefieras. Si no tienes uno, el instructor te pasa el borrador del Momento 3.

PUNTO DE PARTIDA · LA TIENDA DE LA SESIÓN 3 4 MIN

Sigue en tu proyecto, o empieza uno nuevo

terminal

```
# A · Sigue en tu proyecto de la sesión 3 (la Tienda en piezas)
cd componentes
npm run dev

# B · No lo tienes: proyecto nuevo y pega la carpeta src/ del instructor
npm create vue@latest herramientas -- --ts --bare
cd herramientas
npm install
npm run dev
```

Camino A: si tu Tienda de la sesión 3 funciona, sigue en ese proyecto. **Camino B:** si no, crea uno nuevo y reemplaza su `src/` con la carpeta que te pasa el instructor.

La página muestra el resultado. Las DevTools, el estado

PESTAÑA	QUÉ MUESTRA	LA USAMOS PARA…
`Components`	el árbol de componentes, con props y estado	ver `descuento`, `total`, `categoria`
`Timeline`	clics, teclado, eventos de componentes, rendimiento	seguir los emits en orden
`Pages`, `Assets`, `Graph`, `Inspect`	rutas, archivos, módulos y la transformación de cada archivo	—

Se abren con `Alt+Shift+D` (plugin de Vite, ya incluido en tu proyecto) o con F12 → pestaña **Vue** si instalaste la extensión.

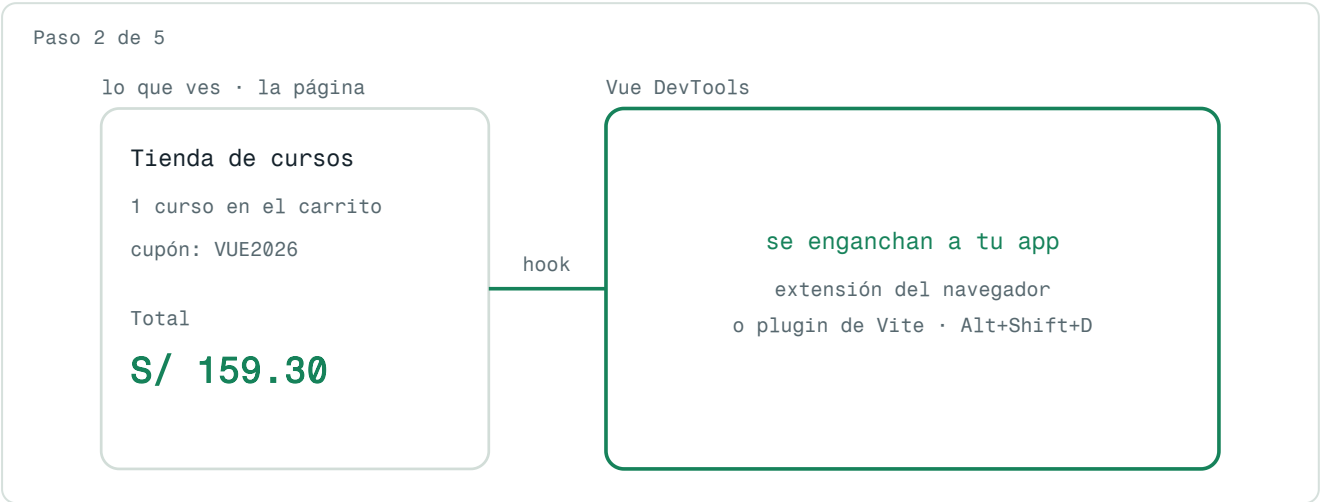
CONCEPTO Vue DevTools

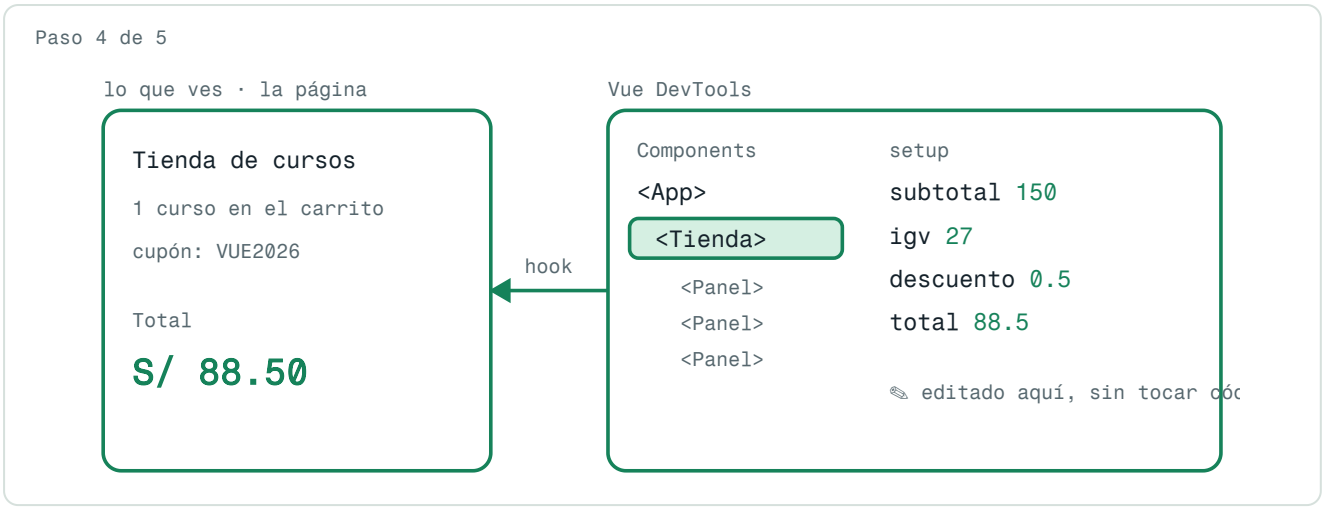
QUÉ ES

Un panel que se engancha a tu app y muestra lo que la página no dice: el **árbol de componentes** y el estado de cada uno.

PARA QUÉ SE USA

Ver props, `ref` y `computed` en vivo, editarlos sin tocar el código y seguir los eventos en el tiempo.





Cómo funciona, paso a paso

1. La página solo muestra el resultado: **S/ 159.30**. Los `ref`, las props y los `computed` que lo producen no se ven.
2. Las DevTools se enganchan a la app: como **extensión del navegador** o con `vite-plugin-vue-devtools`, que ya está en tu proyecto (`Alt+Shift+D`).
3. **Components**: el árbol de componentes. Al elegir `Tienda` ves su estado: `subtotal`, `igv`, `descuento`, `total`.
4. Los valores se pueden **editar**. Cambia `descuento` y la pantalla reacciona: es la reactividad que ya conoces, vista desde afuera.
5. **Timeline**: lo que pasó y en qué orden. Clics, teclado, **eventos de componentes** (los emits) y rendimiento.

REGLA

Lo que ves en la página es el **resultado**. Las DevTools te muestran el **estado** que lo produjo. Se instalan una vez y no cambian tu código.

CONCEPTO

Depurar el estado

QUÉ ES

Un método para encontrar un bug mirando el **estado** antes de leer el código: del síntoma hasta quién es dueño del dato.

PARA QUÉ SE USA

Dejar de adivinar con `console.log`: en las DevTools ves todos los valores a la vez, y cambian cuando cambian en tu app.

Paso 3 de 5

síntoma

Total mostrado

S/ 174.30

esperaba S/ 159.30

DevTools · Components · Tienda

subtotal	150	150 + 27 = 177
igv	27	177 − 10% = 159.3
descuento	0.1	
total (computed)	174.3 ≠ 159.3	

Paso 5 de 5

síntoma

Total mostrado

S/ 159.30

✔ coincide con lo esperado

DevTools · Components · Tienda

subtotal	150	150 + 27 = 177
igv	27	177 − 10% = 159.3
descuento	0.1	
total (computed)	159.3 = 159.3 ✔	

`(subtotal + igv) * (1 − descuento)`

el total vuelve a ser 159.3

Cómo funciona, paso a paso

1. **El síntoma:** el total dice S/ 174.30 y a mano da S/ 159.30. Todavía no sabes si el error es de datos o de cálculo.
2. **Components → Tienda.** En `setup` están todos los `ref` y `computed`, con su valor de ahora mismo.
3. **Compara.** `subtotal` 150 y `igv` 27 suman 177; con 10% de descuento deberían dar 159.3. El `total` dice 174.3.
4. **Decide dónde buscar.** Las entradas están bien y la salida mal: el error está en la fórmula del `computed`. Si una entrada viniera mal, subirías al dueño del dato.
5. **Comprueba.** Corrige el `computed`: Vite recarga y el `total` de las DevTools vuelve a cuadrar. Sin un solo `console.log`.

REGLA

Si las **entradas están bien** y el resultado mal, falla quien calcula. Si una **entrada ya viene mal**, sube al dueño del dato: el padre, o el composable.

Momento 1 de 3 · Depurar la Tienda

· 8 min

Un «compañero» cambió la fórmula del total. Lo encontramos mirando el estado, sin leer todo el código.

1. **Components → Tienda** — agrega el primer curso y mira en `setup`: `subtotal`, `igv`, `descuento`, `total`. Elige una `TarjetaCurso` (props y `abierto`) y un `ItemCarrito`.
2. **Components → Tienda** — cambia `descuento` a `0.5`: el total baja sin escribir el cupón. Recarga: todo vuelve a como estaba.
3. **Tienda · Script** — reemplaza el `computed` del total por la versión del compañero (abajo) y prueba el cupón `VUE2026`.
4. **Components → Tienda** — compara: `subtotal` 150 + `igv` 27 = 177; con 10% de descuento, 159.3. El `total` dice 174.3. Las entradas están bien, falla la fórmula: corrígela.
5. **Timeline** — ⏭ activa la capa *Component events*, graba y pulsa «Agregar»: verás el emit.

```
// Lo que cambió el «compañero» en Tienda.vue
// antes
const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))
// después: el descuento solo afecta al IGV
const total = computed(() => subtotal.value + igv.value * (1 - descuento.value))
```

EL MÉTODO

1. El síntoma. **2.** Abre el componente y mira su estado. **3.** Compara entradas y salida. **4.** ¿Falla el dato o el cálculo? Si es el dato, sube al dueño. **5.** Comprueba la corrección.

Tres guardias, tres preguntas distintas

HERRAMIENTA	PREGUNTA	ARCHIVO DE CONFIGURACIÓN	SCRIPT
vue-tsc	¿los tipos cuadran?	tsconfig.app.json	type-check
ESLint	¿hay errores o malas prácticas?	eslint.config.ts	lint
Prettier	¿se ve como el resto?	.prettierrc.json	format, format:check

CONCEPTO Tres guardias

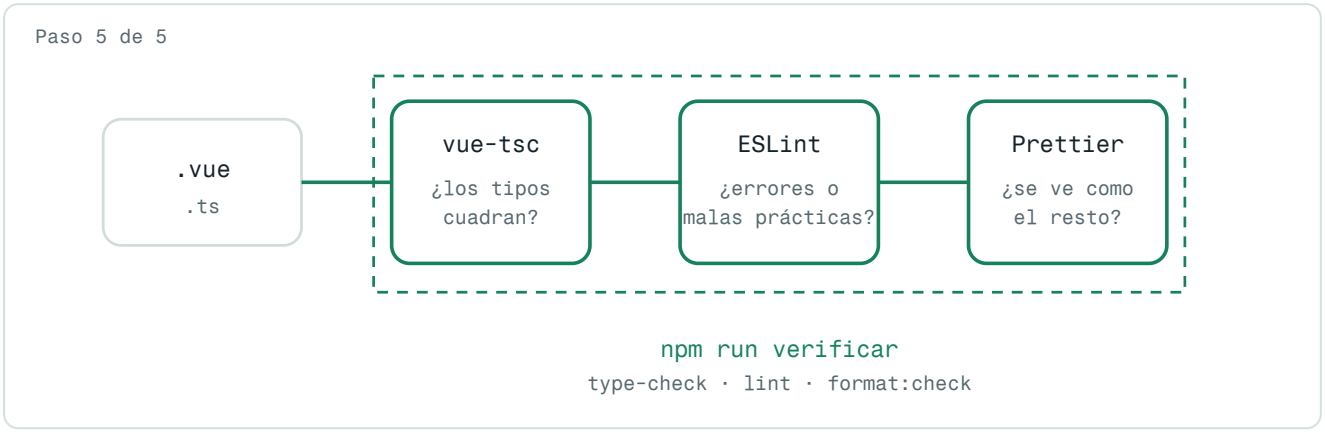
QUÉ ES

Tres herramientas que **revisan tu código sin ejecutarlo**. Cada una hace una pregunta distinta.

PARA QUÉ SE USA

Encontrar los errores antes de que lleguen al navegador, y que todo el equipo escriba igual.





Cómo funciona, paso a paso

1. Tu código: archivos `.vue` y `.ts`. Funciona en el navegador, pero eso no dice que esté bien hecho.
2. **vue-tsc** pregunta: ¿los **tipos cuadran**? Revisa props, emits y las expresiones del template.
3. **ESLint** pregunta: ¿hay **errores o malas prácticas**? Variables sin usar, `v-for` sin `:key`, `any`…
4. **Prettier** pregunta: ¿**se ve como el resto**? Comillas, comas, saltos de línea. Es una máquina, no una opinión.
5. Los tres caben en un solo comando: `npm run verificar`. En el editor avisan mientras escribes; en el repositorio, antes de integrar.

REGLA

Una herramienta, un trabajo: **tipos**, **errores**, **formato**. Si una hace el trabajo de otra, se pelean.

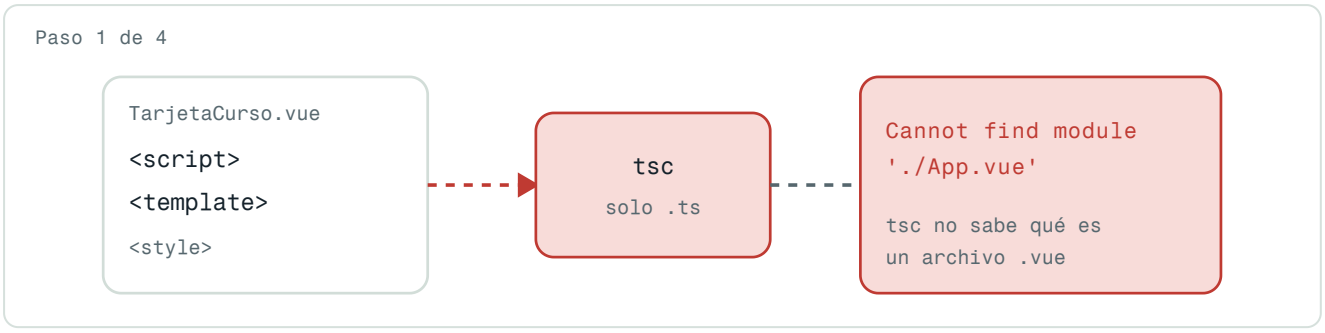
CONCEPTO vue-tsc

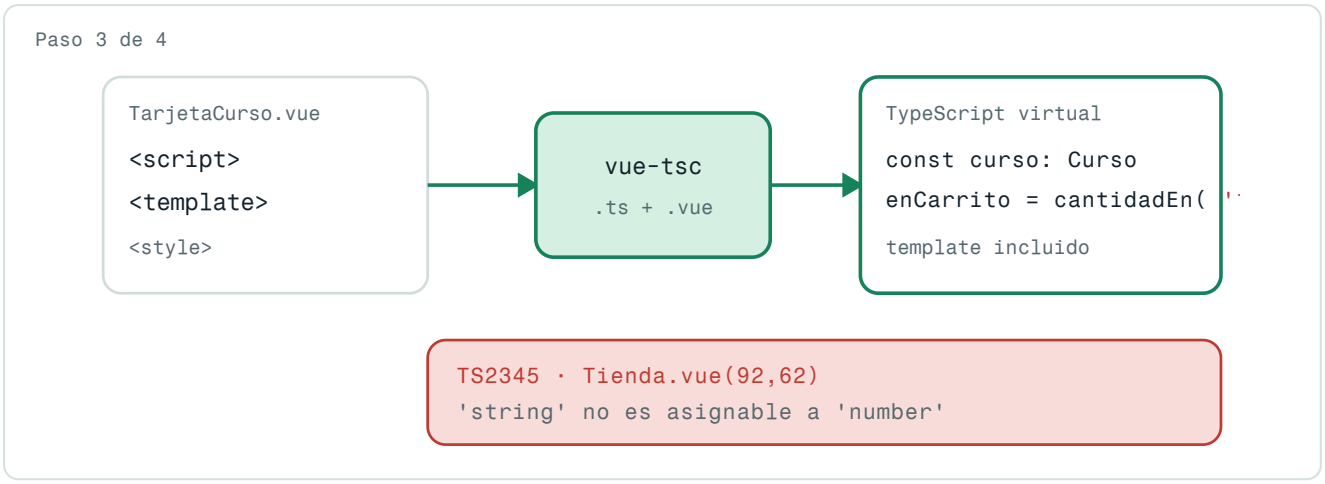
QUÉ ES

El compilador de TypeScript, pero que **entiende los archivos** `.vue`: el script y también el template.

PARA QUÉ SE USA

Comprobar que las props, los emits y cada expresión del template respetan sus tipos, en todo el proyecto y no solo en el archivo abierto.





Cómo funciona, paso a paso

1. `tsc` solo lee `.ts`. Con un `.vue` dice *Cannot find module './App.vue'*: no sabe qué es.
2. `vue-tsc` convierte cada `.vue` en **TypeScript virtual** (script y template juntos) y lo revisa con el mismo compilador.
3. Por eso ve el template: `cantidadEn('1')` con un string da **TS2345**. Sin `vue-tsc`, ese error aparecería recién en el navegador.
4. En el editor lo hace **Vue (Official)**, con subrayados en vivo. En la terminal: `vue-tsc --build`, que es el script `type-check` de tu proyecto.

REGLA

El editor te avisa **archivo por archivo**; `vue-tsc` revisa **todo el proyecto**. `npm run build` ya lo corre, y un error de tipos detiene el build.

CONCEPTO

ESLint · flat config

QUÉ ES

Un **linter**: lee tu código y aplica reglas que detectan errores y malas prácticas. Se configura en `eslint.config.ts` (flat config).

PARA QUÉ SE USA

Atrapar lo que los tipos no ven: variables sin usar, `v-for` sin `:key`, mutar una prop, `any`.

Paso 3 de 5

eslint.config.ts · export default […]

```
{ files: ['**/*.{vue,ts}'] }
...pluginVue.configs['flat/essential']
vueTsConfigs.recommended
```

```
$ eslint .
Panel.vue 1:1 error
Tienda.vue 1:1 error
multi-word-component-names
✖ 2 problems
```

Paso 4 de 5

eslint.config.ts · export default […]

```
{ files: ['**/*.{vue,ts}'] }
...pluginVue.configs['flat/essential']
vueTsConfigs.recommended
{ rules: { 'vue/multi-word-…': … } }
```

← el último gana

```
$ eslint .
✔ sin errores
ignores: ['Tienda', 'Panel']
```

Cómo funciona, paso a paso

1. Un solo archivo, `eslint.config.ts`, exporta un **arreglo de objetos**. Ya no hay `.eslintrc` ni `extends`.
2. Se arma con **capas**: qué archivos revisar, los presets de Vue (`flat/essential`) y de TypeScript (`recommended`). Cada preset trae decenas de reglas.
3. Corremos `eslint .` y nuestro propio código falla: `Tienda` y `Panel` son nombres de **una palabra** (`multi-word-component-names`).
4. No cambiamos el preset: agregamos un **objeto propio al final** con esa regla y sus excepciones. Como el último gana, el error desaparece.
5. `skipFormatting` va al final: apaga las reglas de ESLint que discuten el formato, para que se encargue solo Prettier.

REGLA

La flat config es **un arreglo de objetos** y el orden importa: **el último gana**. Para cambiar una regla, agrega un objeto al final.

CONCEPTO

Prettier

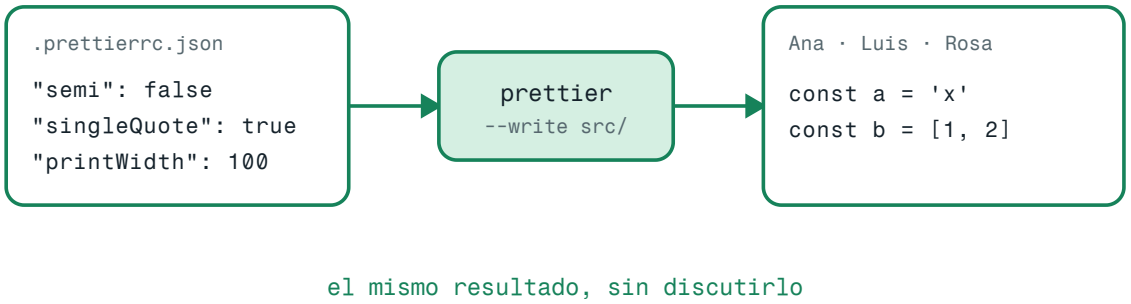
QUÉ ES

Un **formateador** con opiniones: reescribe tu código con un único estilo, definido en `.prettierrc.json`.

PARA QUÉ SE USA

Acabar con las discusiones de estilo y con los cambios que ensucian cada *pull request*: nadie formatea a mano.

Paso 2 de 5



Paso 5 de 5



Cómo funciona, paso a paso

1. Sin acuerdo, cada persona escribe distinto: comillas, `;`, espacios. El resultado son diffs con ruido y discusiones que no aportan.
2. Una config con tres opciones (`semi`, `singleQuote`, `printWidth`) y `prettier --write`: **todos quedan igual**.
3. Solo cambia el **formato**: en la Tienda, une una línea larga y parte un `<strong>`. Lo que hace el código es idéntico.
4. Se corre en tres momentos: **al guardar** en el editor, **antes del commit**, y en CI con `--check`, que falla si algo no está formateado.
5. **ESLint revisa, Prettier formatea.** `eslint-config-prettier` (`skipFormatting`) apaga las reglas de ESLint que pelean con Prettier.

REGLA

El formato **no se discute, se automatiza**. Prettier solo cambia cómo se ve el código, nunca lo que hace; la revisión de errores es de ESLint.

Momento 2 de 3 · Los tres guardias en la Tienda

· 10 min

Instalamos ESLint y Prettier, hacemos fallar a cada guardia a propósito y dejamos todo en verde.

1. **Terminal** — instala ESLint y Prettier (`--bare` solo trae `vue-tsc`). Crea `eslint.config.ts` y `.prettierrc.json` en la raíz y agrega los scripts `lint`, `format`, `format:check` y `verificar`.
2. **Terminal** — `npm run type-check` pasa. **Tienda · Template** — cambia `cantidadEn(curso.id)` por `cantidadEn('1')`: error TS2345, aunque esté en el template. Deshazlo.
3. **Terminal** — `npm run lint`: 2 errores, `Tienda` y `Panel` son de una palabra. **eslint.config.ts** — agrega un objeto propio al final con `'vue/multi-word-component-names'` y `ignores`.
4. **Terminal** — `npm run format`: `Tienda.vue` cambia en dos sitios. `npm run verificar` corre los tres guardias seguidos.
5. **VS Code** — ⏭ activa *Format On Save* con Prettier como formateador.

terminal

```
npm install -D eslint @eslint/js eslint-plugin-vue @vue/eslint-config-typescript eslint-
   config-prettier jiti prettier
```

eslint.config.ts

```ts
import { globalIgnores } from 'eslint/config'
import { defineConfigWithVueTs, vueTsConfigs } from '@vue/eslint-config-typescript'
import pluginVue from 'eslint-plugin-vue'
import skipFormatting from 'eslint-config-prettier/flat'

export default defineConfigWithVueTs(
  { name: 'app/files-to-lint', files: ['**/*.{vue,ts}'] },

  globalIgnores(['**/dist/**', '**/coverage/**']),

  ...pluginVue.configs['flat/essential'],
  vueTsConfigs.recommended,

  {
    name: 'app/reglas-propias',
    rules: {
      // Tienda y Panel son nombres de una palabra a propósito
      'vue/multi-word-component-names': ['error', { ignores: ['Tienda', 'Panel'] }],
    },
  },

  skipFormatting,
)
```

.prettierrc.json

```
{
  "semi": false,
  "singleQuote": true,
  "printWidth": 100
}
```

package.json

```
"scripts": {
  "type-check": "vue-tsc --build",
  "lint": "eslint .",
  "format": "prettier --write src/",
  "format:check": "prettier --check src/",
  "verificar": "run-s type-check lint format:check"
}
```

BLOQUE 3 · ASISTENTES DE IA EN EL EDITOR 20 MIN

La IA propone. Las herramientas y tú deciden

MODO	CÓMO SE USA	QUÉ REVISAR
Autocompletado	sugiere mientras escribes; Tab para aceptar	cada línea que aceptas
Chat o edición en línea	le pides algo concreto y propone un cambio	el cambio completo antes de aceptarlo
Agente	lee archivos, edita y corre comandos	todo el diff y lo que ejecutó

CONCEPTO IA en el editor

QUÉ ES

Un modelo de lenguaje **dentro del editor** que escribe código contigo. Se usa de tres formas, de menos a más autonomía.

PARA QUÉ SE USA

Ir más rápido en lo repetitivo (un componente base, un test, un patrón que ya existe) y dejar tu atención para lo que decide.





Cómo funciona, paso a paso

1. **Autocompletado.** Sugiere mientras escribes y aceptas con `Tab`. Lee el archivo abierto: acierta cuando hay un patrón que seguir.

2. **Chat o edición en línea.** Le pides algo concreto («crea un filtro por categoría») y propone un cambio que **aceptas o rechazas**.
3. **Agente.** Lee varios archivos, edita, corre comandos y vuelve con un resultado. Hace más, así que también hay más cambios que leer.
4. De un extremo al otro, la autonomía crece… y **la revisión también**. Elige el modo según cuánto puedas verificar.

REGLA

La IA acelera, pero **tú respondes por el código**. Cuanta más autonomía le das, más tienes que revisar.

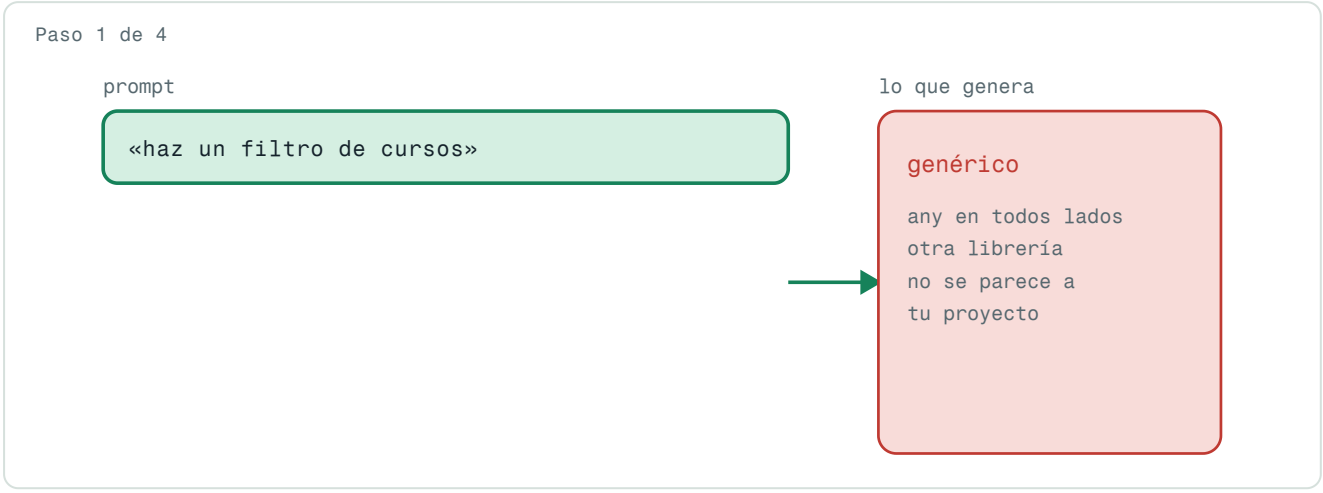
CONCEPTO Contexto + contrato

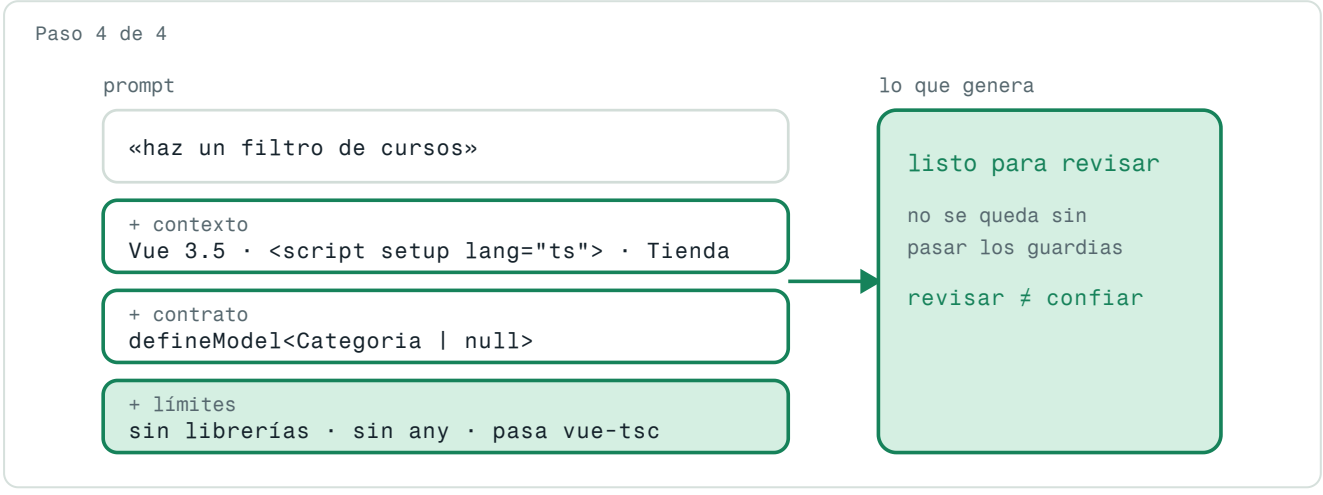
QUÉ ES

Una petición que incluye lo que la IA **no puede adivinar**: tu **contexto** (stack y archivos vecinos), tu **contrato** (los tipos) y tus **límites**.

PARA QUÉ SE USA

Que el componente generado se parezca a los de tu proyecto y no a una respuesta genérica de internet.





Cómo funciona, paso a paso

1. **Prompt vago:** «haz un filtro de cursos». La IA rellena los huecos con lo más común: `any`, otra librería, otro estilo.
2. **+ Contexto:** Vue 3.5, `<script setup lang="ts">` y los archivos vecinos (`Tienda`, `Panel`). Ya sigue tu stack, pero inventa los tipos.
3. **+ Contrato:** el tipo `Curso` y la firma que quieres, `defineModel<Categoria | null>`. Ahora usa tus tipos y las mismas piezas que el resto.
4. **+ Límites:** sin librerías, sin `any`, que pase `vue-tsc` y ESLint. Sale algo **listo para revisar**, no para confiar.

REGLA

Dale los **tipos que ya tienes**: son la forma más corta de decirle lo que quieres, y `vue-tsc` comprueba después que los respetó.

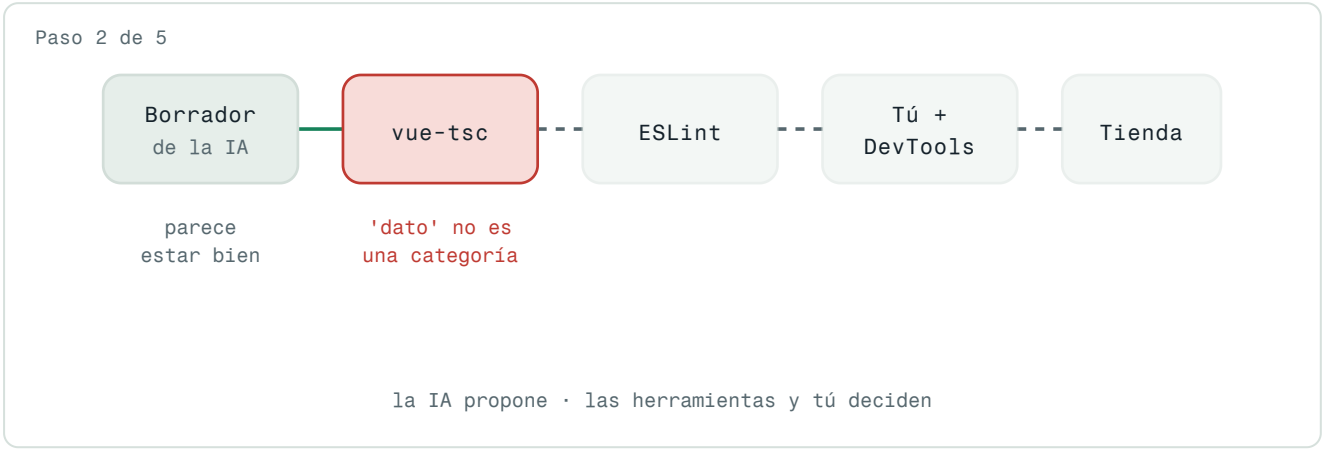
CONCEPTO Revisar lo generado

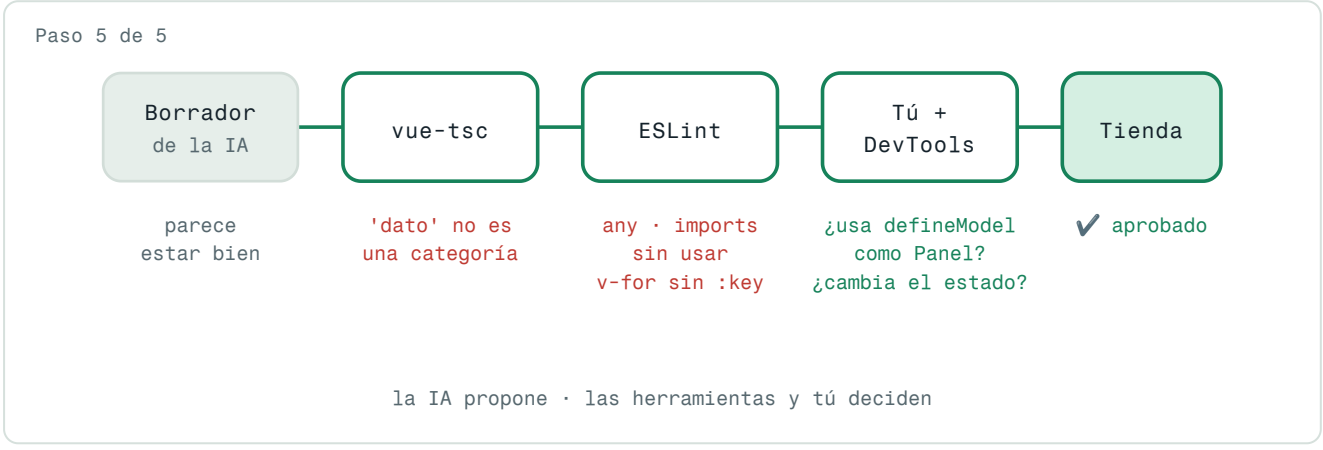
QUÉ ES

Tratar el código generado como el de **cualquier compañero nuevo**: pasa por los mismos guardias y por tus ojos antes de entrar al proyecto.

PARA QUÉ SE USA

Aprovechar la velocidad de la IA sin heredar sus errores: lo que parece correcto no siempre lo es.





Cómo funciona, paso a paso

1. La IA entrega un **borrador** de `FiltroCategoria`. Se ve bien y hasta funcionaría… pero todavía no sabemos si está bien hecho.
2. **vue-tsc** lo frena: la categoría `'dato'` no existe, es `'datos'`. Un error que en el navegador pasaría desapercibido.
3. **ESLint** encuentra lo suyo: un `any`, imports que no se usan y un `v-for` sin `:key`.
4. **Tú** miras lo que ninguna herramienta ve: ¿usa `defineModel` como el resto de la Tienda? Y las **DevTools** confirman que el estado cambia al hacer clic.
5. Corregido y en verde, **entra a la Tienda**. Lo que hace falta para aprobarlo es lo mismo que para aprobar tu propio código.

> REGLA
>
> La IA **propone**; las herramientas y tú **deciden**. Si no pasa `vue-tsc` y ESLint, no se queda, venga de quien venga.

Momento 3 de 3 · Generar y revisar FiltroCategoria

▬ · 8 min

Le pedimos a la IA un componente, lo pasamos por los tres guardias y lo conectamos a la Tienda.

1. **Tu asistente de IA** — pega el prompt (abajo) con `Tienda.vue`, `Panel.vue` y `cursos.ts` adjuntos. No aceptes la respuesta todavía.
2. **Archivo nuevo** — `src/components/FiltroCategoria.vue` con el borrador del instructor (abajo). Cada asistente responde distinto: el borrador es el mismo para todos.
3. **Terminal** — `npm run type-check` (la categoría `'dato'` no existe) y `npm run lint` (cinco errores: `any`, imports y variable sin usar, `v-for` sin `:key`).
4. **FiltroCategoria** — reemplázalo por la versión revisada (abajo): usa `defineModel` como el resto de la Tienda y marca el filtro activo. `npm run verificar` pasa.
5. **Tienda** — ⏭ importa el componente, agrega `categoria` y un `filter` en `visibles`, y `<FiltroCategoria v-model="categoria" />` sobre el buscador. Confírmalo en las DevTools.

prompt

```
Crea src/components/FiltroCategoria.vue para mi Tienda de cursos.

Contexto: Vue 3.5, <script setup lang="ts">, TypeScript estricto.
Sigue el estilo de Panel.vue y ListaEstados.vue (adjuntos).

Contrato:
- El tipo Curso está en src/cursos.ts: su categoria es
  'frontend' | 'backend' | 'datos'.
- v-model con defineModel<Categoria | null>({ required: true });
  null significa «todas las categorías».
- Un botón por categoría más uno «Todas», con aria-pressed.

Límites: sin librerías, sin any, sin CSS. Debe pasar vue-tsc y ESLint.
```

src/components/FiltroCategoria.vue · borrador

```vue
<script setup lang="ts">
import { ref, computed } from 'vue'
import type { Curso } from '@/cursos'

const props = defineProps<{ modelValue: any }>()
const emit = defineEmits(['update:modelValue'])

const categorias: Curso['categoria'][] = ['frontend', 'backend', 'dato']
</script>

<template>
  <div>
    <button @click="emit('update:modelValue', null)">Todas</button>
    <button v-for="c in categorias" @click="emit('update:modelValue', c)">{{ c }}</button>
  </div>
</template>
```

src/components/FiltroCategoria.vue · revisado

```vue
<script setup lang="ts">
import type { Curso } from '@/cursos'

type Categoria = Curso['categoria']

const categoria = defineModel<Categoria | null>({ required: true })

const categorias: Categoria[] = ['frontend', 'backend', 'datos']
</script>

<template>
  <div role="group" aria-label="Filtrar por categoría">
    <button type="button" :aria-pressed="categoria === null" @click="categoria = null">
      Todas
    </button>
    <button
      v-for="c in categorias"
      :key="c"
      type="button"
      :aria-pressed="categoria === c"
      @click="categoria = c"
    >
      {{ c }}
    </button>
  </div>
</template>
```

LA TIENDA, REVISADA · CHECKPOINT 3

La Tienda terminada

La Tienda con el filtro por categoría, ya formateada y sin errores en **los tres guardias**.

src/components/Tienda.vue

```
<script setup lang="ts">
import { ref, computed, watch, watchEffect, useTemplateRef, onMounted } from 'vue'
import { CURSOS, type Curso } from '@/cursos'
import { useCarrito } from '@/composables/useCarrito'
import { useLocalStorage } from '@/composables/useLocalStorage'
import ItemCarrito from './ItemCarrito.vue'
import TarjetaCurso from './TarjetaCurso.vue'
import Panel from './Panel.vue'
import ListaEstados from './ListaEstados.vue'
import CampoTexto from './CampoTexto.vue'
import FiltroCategoria from './FiltroCategoria.vue'

const { carrito, agregar, quitar, vaciar, cantidadEn, cantidadTotal, subtotal, igv } =
   useCarrito()

const buscador = useTemplateRef('buscador')

function vaciarYBuscar() {
  vaciar()
  buscador.value?.focus()
}

const cupon = ref('')
const mensaje = ref('')
const descuento = ref(0)

const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))

watch(cupon, (nuevo, anterior) => {
  console.log(`cupón: "${anterior}" → "${nuevo}"`)
  mensaje.value = 'Validando…'
  setTimeout(() => {
    const valido = nuevo === 'VUE2026'
    descuento.value = valido ? 0.1 : 0
    mensaje.value = valido ? '✅ 10% de descuento' : '❌ Cupón no válido'
  }, 800)
})

watchEffect(() => {
  document.title = `Tienda (${cantidadTotal.value})`
})

const cargando = ref(true)
const cursos = ref<Curso[]>([])

setTimeout(() => {
  cursos.value = CURSOS
  cargando.value = false
}, 1500)

const busqueda = ref('')
const categoria = ref<Curso['categoria'] | null>(null)
const orden = ref<'titulo' | 'precio'>('titulo')
```

```
const visibles = computed(() => {
  const texto = busqueda.value.toLowerCase()
  return cursos.value
    .filter((c) => c.titulo.toLowerCase().includes(texto))
    .filter((c) => categoria.value === null || c.categoria === categoria.value)
    .sort((a, b) =>
      orden.value === 'precio' ? a.precio - b.precio : a.titulo.localeCompare(b.titulo),
    )
})

const cliente = useLocalStorage('tienda-cliente', { nombre: '', email: '' })

const campoNombre = useTemplateRef('campoNombre')

onMounted(() => campoNombre.value?.enfocar())
</script>

<template>
  <h1>Tienda de cursos</h1>
  <p>Pedido de {{ cliente.nombre }} · {{ cantidadTotal }} cursos</p>

  <Panel titulo="Alumno">
    <CampoTexto ref="campoNombre" v-model="cliente.nombre" etiqueta="Nombre" />
    <CampoTexto v-model="cliente.email" etiqueta="Email" />
  </Panel>

  <Panel titulo="Catálogo">
    <template #acciones>
      <span v-if="!cargando">{{ visibles.length }} cursos</span>
    </template>

    <FiltroCategoria v-model="categoria" />
    <input ref="buscador" v-model="busqueda" placeholder="Buscar curso…" />
    <select v-model="orden">
      <option value="titulo">Por título</option>
      <option value="precio">Por precio</option>
    </select>

    <ListaEstados :items="visibles" :clave="(curso) => curso.id" :cargando="cargando">
      <template #item="{ item: curso }">
        <TarjetaCurso :curso="curso" :en-carrito="cantidadEn(curso.id)" @agregar="agregar" />
      </template>
      <template #vacio>
        <p>No hay cursos con "{{ busqueda }}".</p>
      </template>
    </ListaEstados>
  </Panel>

  <Panel titulo="Carrito">
    <template #acciones>
      <button @click="vaciarYBuscar">Vaciar</button>
    </template>

    <ListaEstados :items="carrito" :clave="(linea) => linea.curso.id">
      <template #item="{ item: linea }">
        <ItemCarrito
          v-model:cantidad="linea.cantidad"
          :titulo="linea.curso.titulo"
          :precio="linea.curso.precio"
          @quitar="quitar(linea.curso.id)"
        />
      </template>
      <template #vacio>
```

```
        <p>Tu carrito está vacío.</p>
      </template>
    </ListaEstados>

    <input v-model.lazy="cupon" placeholder="Cupón de descuento" />
    <p>{{ mensaje }}</p>

    <template #pie>
      <p>Subtotal: S/ {{ subtotal.toFixed(2) }}</p>
      <p>IGV (18%): S/ {{ igv.toFixed(2) }}</p>
      <p>
        <strong>Total: S/ {{ total.toFixed(2) }}</strong>
      </p>
    </template>
  </Panel>
</template>
```

CIERRE · PARA SEGUIR

Ocho decisiones de herramienta

NECESITO…	USO
ver el estado y las props de un componente	DevTools · Components
saber qué eventos pasaron y en qué orden	DevTools · Timeline
que los tipos cuadren, template incluido	`vue-tsc` (`type-check`)
encontrar errores y malas prácticas	ESLint · `eslint.config.ts`
que todo el equipo escriba igual	Prettier · `.prettierrc.json`
revisar todo con un solo comando	`npm run verificar`
código nuevo, rápido	IA con contexto + contrato
confiar en lo que generó la IA	pasarlo por los guardias

Para practicar

Agrega a `eslint.config.ts` la regla `no-console` como `'warn'` y limpia el `console.log` del `watch(cupon)`. Bonus: pídele a la IA un componente `OrdenarPor` para el catálogo, con su contexto y su contrato, y revísalo con los tres guardias.

Glosario

TÉRMINO	EN UNA FRASE
Vue DevTools	Panel que muestra el árbol de componentes, su estado y sus eventos.
Depurar	Encontrar la causa de un comportamiento incorrecto, empezando por mirar el estado.
`vue-tsc`	TypeScript que entiende archivos `.vue`, incluido el template.
Linter (ESLint)	Programa que revisa el código con reglas, sin ejecutarlo.
Flat config	`eslint.config.ts`: un arreglo de objetos aplicados en orden; el último gana.
Preset	Un conjunto de reglas ya armado (`flat/essential`, `recommended`).
Prettier	Formateador que reescribe el código con un solo estilo.
Agente	Modo de la IA que lee archivos, edita y corre comandos por su cuenta.
Contexto y contrato	Lo que le das a la IA: tu stack y archivos, y los tipos que debe respetar.

Para repasar

Vue DevTools	devtools.vuejs.org
Herramientas de Vue (tooling)	vuejs.org
TypeScript con la Composition API	vuejs.org
ESLint · flat config	eslint.org
eslint-plugin-vue	eslint.vuejs.org
Prettier · opciones	prettier.io